



DISEÑO DE SOFTWARE 2026

CuriDocs

GUÍA ESENCIAL DEL REPOSITORIO

Del boceto al producto real

Cómo funciona la página actual, dónde vive cada pieza y qué disciplina debe seguir el equipo para continuar el proyecto.

BOCETO NAVEGABLE

SIN LÓGICA DE NEGOCIO

GUÍA 2026

IDEA CENTRAL

Preparado no significa desarrollado.

El repositorio ofrece estructura, navegación y controles de calidad. La autenticación, los datos, los formularios, los documentos y las reglas del dominio siguen siendo responsabilidad del equipo.

Versión 1.0 | 24 de agosto de 2026 | Equipo CuriDocs

Basado en el enunciado preliminar del 13 de agosto de 2026 y el repositorio actual.



Estado actual y mapa de la guía

Esta guía separa con claridad tres cosas: la intención académica, la base técnica que ya existe y el trabajo funcional que todavía debe realizar el grupo.

YA EXISTE Repositorio organizado Web navegable, API mínima, contratos, CI, documentación y PostgreSQL local opcional.	NO EXISTE Producto funcional No hay autenticación, CRUD, modelo de datos, editor, PDF, trazabilidad real ni integraciones.	PRÓXIMO OBJETIVO Incrementos verticales Implementar requisitos aprobados de extremo a extremo, con pruebas y documentación.
---	--	---

CONTENIDO

01	Proyecto académico	Objetivo, etapas y roles	03
02	Producto actual	Pantallas, rutas y comportamiento	04-05
03	Base técnica	Arquitectura, carpetas y comandos	06-08
04	Forma de trabajo	Implementación, Git y decisiones	09-12

REGLA CENTRAL

Todo lo visible hoy es una guía de conversación.

Los botones navegan, pero no guardan. Los indicadores comunican una idea, pero no calculan. Las páginas describen módulos, pero no ejecutan casos de uso.

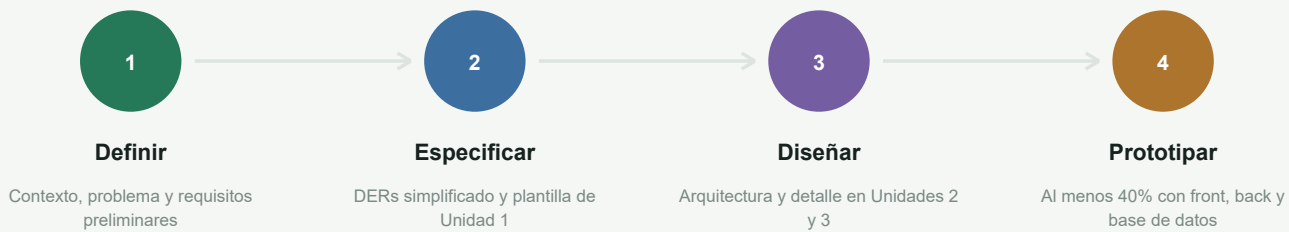


El proyecto académico en una página

OBJETIVO

Definir, especificar, diseñar y prototipar un sistema que gestione los documentos generados en cada fase de un proceso de software tradicional.

ETAPAS EVALUADAS



ROLES POR UNIDAD



Arquitectura

Coordina la visión y las decisiones estructurales.



Análisis y diseño

Mantiene requisitos, criterios y trazabilidad.



UI/UX y diseño

Define interacción, accesibilidad y validación.



Desarrollo front/back

Implementa incrementos y pruebas del sistema.

ADVERTENCIA DE ALCANCE

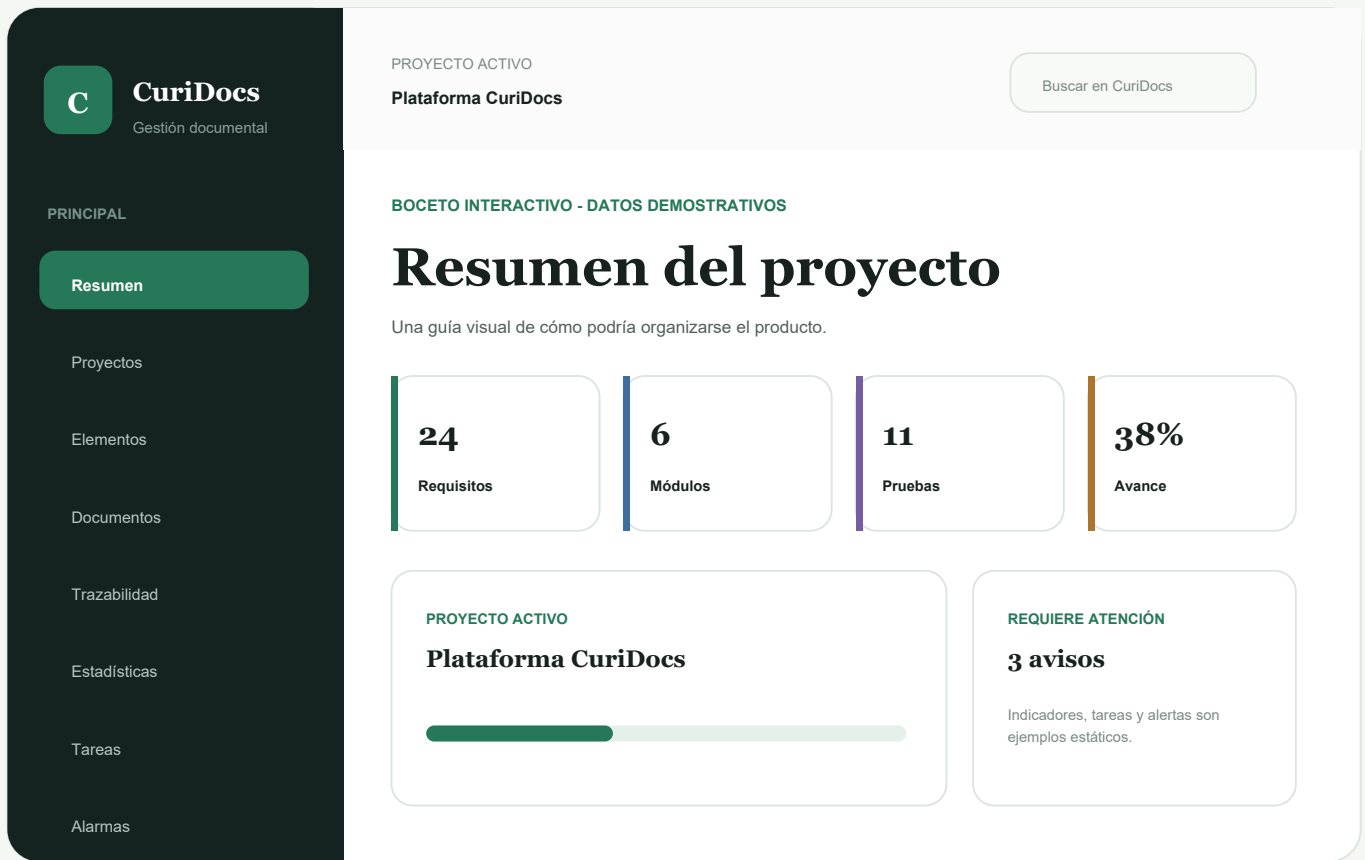
El enunciado y el modelo de datos son preliminares.

Las aclaraciones oficiales y las plantillas de cada unidad tienen prioridad. No conviertan una idea preliminar en código irreversible sin validarla y registrarla.



Cómo funciona la página actualmente

VISTA CONCEPTUAL DEL TABLERO



FLUJO DE INTERACCIÓN



QUÉ SÍ SE PUEDE HACER

- Recorrer el menú lateral y la barra superior.
- Abrir tarjetas, alertas y tareas de muestra.
- Llegar a una guía específica por módulo.

QUÉ TODAVÍA NO OCURRE

- No se guardan ni editan datos.
- No existe sesión, permiso ni usuario real.
- No hay llamadas de dominio ni persistencia.



Mapa de navegación y rutas

Todas las rutas siguientes se generan a partir de un catálogo central. Cada una presenta alcance tentativo y próximos pasos, no una funcionalidad terminada.

PR Proyectos /proyectos Organización, responsables e hitos	EL Elementos /elementos Requisitos, módulos y casos de prueba
DO Documentos /documentos Biblioteca, versiones y revisión	TR Trazabilidad /trazabilidad Relaciones, matrices y cobertura
ES Estadísticas /estadisticas Indicadores y gráficos futuros	TA Tareas /tareas Pendientes, prioridad y responsables
EQ Equipo /equipo Integrantes, roles y permisos	AL Alarmas /alarmas Inconsistencias y vencimientos
BU Buscar /buscar Exploración global y filtros	NO Notificaciones /notificaciones Actividad personal
AY Ayuda /ayuda Primeros pasos y glosario	CO Configuración /configuracion Preferencias del espacio

PIEZAS CLAVE

Catálogo

_data/navigation.ts

Plantilla dinámica

[section]/page.tsx

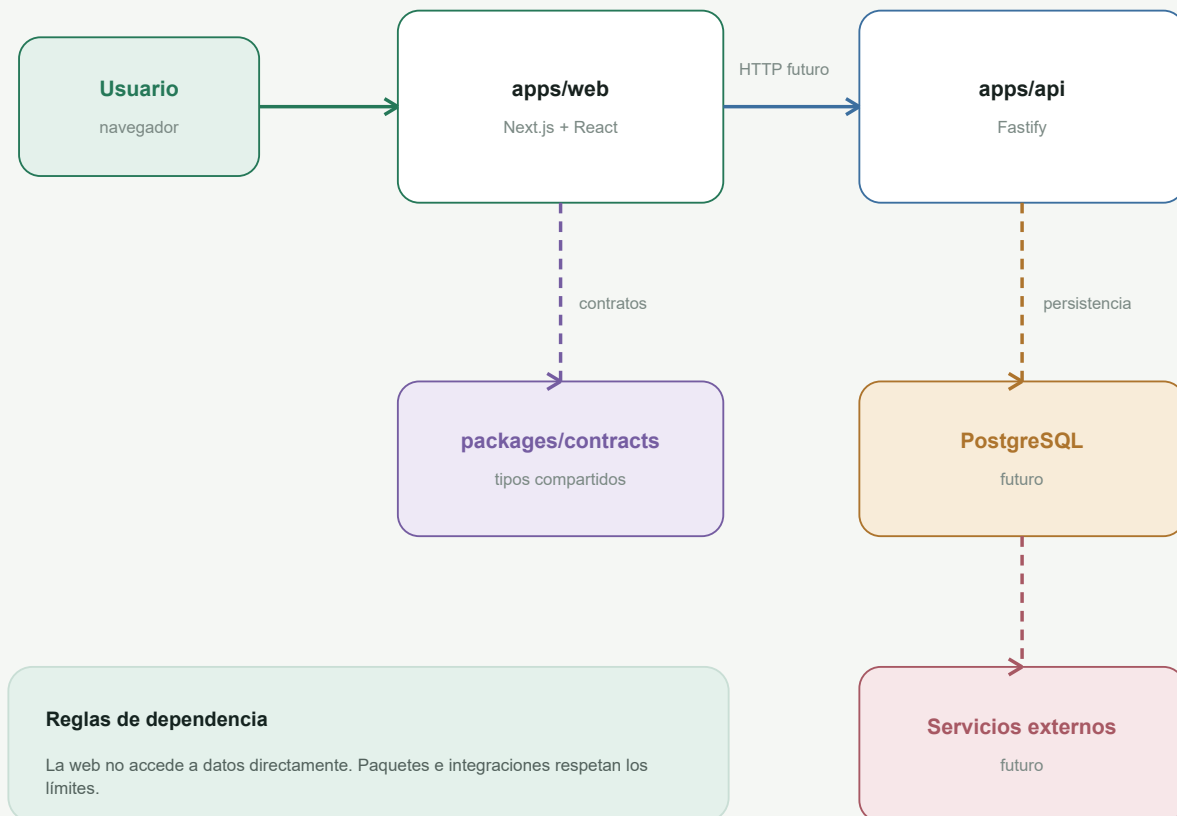
Carcasa persistente

_components/app-shell.tsx



Arquitectura técnica actual

La estructura separa interfaz, servicio HTTP, contratos compartidos y persistencia futura. Las líneas punteadas indican capacidades preparadas pero aún no conectadas.



ESTADO DE CADA LÍMITE

Web	Boceto responsive, navegación y GET /api/health.	ACTIVO
API	Servidor mínimo con GET /health. Sin dominio.	ACTIVO
Contratos	Contrato de salud. Sin entidades del producto.	MÍNIMO
Base de datos	Contenedor opcional. Sin tablas ni conexión.	RESERVADO



Carpetas y archivos importantes

ÁRBOL PRINCIPAL

```
CuriDocs/

|- apps/

|   |- web/

|   |   `-- src/app/

|   `-- api/

|       `-- src/

|- packages/

|   `-- contracts/

|- docs/

|   |- adr/

|   `-- requirements/

|- infra/database/

|- .github/

|- compose.yaml

|- package.json

`-- turbo.json
```

ARCHIVOS QUE ORIENTAN EL TRABAJO

- [README.md](#)
Entrada al repositorio y comandos.
- [docs/SCOPE.md](#)
Límite entre preparación y producto.
- [docs/ARCHITECTURE.md](#)
Dirección de dependencias.
- [docs/UI-GUIDE.md](#)
Alcance del boceto actual.
- [CONTRIBUTING.md](#)
Flujo Git y definición de terminado.
- [docs/adr/](#)
Decisiones técnicas justificadas.
- [docs/requirements/](#)
Convenciones para requisitos y trazado.

DÓNDE MODIFICAR LA INTERFAZ

page.tsx	tablero principal
app-shell.tsx	menú y barra superior
navigation.ts	rutas y textos de módulos
[section]/page.tsx	plantilla en construcción
globals.css	estilos y responsive

REGLA PRÁCTICA: UBICAR EL CAMBIO POR RESPONSABILIDAD

Interfaz	Caso de uso	Contrato	Persistencia
apps/web	apps/api	packages/contracts	infra/database
Pantallas, navegación, estados visuales.	Validación, autorización y coordinación.	Tipos compartidos entre aplicaciones.	Migraciones reproducibles, cuando se elija herramienta.



Puesta en marcha y comandos

El boceto y la API pueden iniciarse sin PostgreSQL. La base de datos es opcional hasta que el equipo apruebe el modelo y la herramienta de migraciones.

REQUISITOS

Node.js

24.12 o superior

pnpm

11.19.0

Docker

solo para PostgreSQL

PRIMERA INSTALACIÓN

```
01 corepack enable
02 pnpm install
03 Copy-Item .env.example .env
04 pnpm doctor
05 pnpm dev
```

DIRECCIONES LOCALES

WEB

http://localhost:3000

SALUD WEB

/api/health

SALUD API

http://localhost:3001/health

COMANDOS HABITUALES

pnpm dev

Inicia web y API.

pnpm dev:web

Inicia solo Next.js.

pnpm dev:api

Inicia solo Fastify.

pnpm format

Aplica formato con Prettier.

pnpm check

Formato, lint, tipos, pruebas y build.

pnpm db:up

Levanta PostgreSQL opcional.

pnpm db:logs

Observa el contenedor de datos.

pnpm db:down

Detiene la base sin borrar volumen.

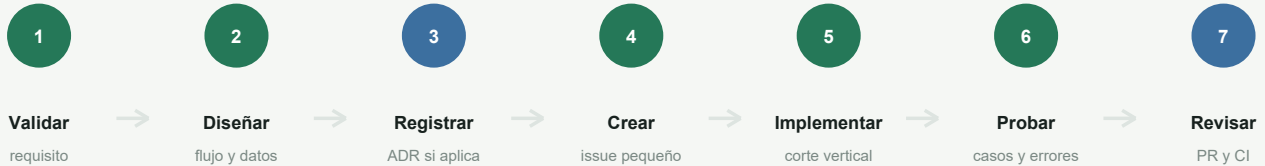
Antes de subir cambios: ejecutar pnpm check y resolver todos los errores. No versionar .env, claves, tokens ni datos reales de estudiantes.



Cómo desarrollar un módulo real

No reemplacen una página en construcción por muchas capas a la vez. Elijan una historia pequeña y llévenla completa desde la interfaz hasta la prueba.

FLUJO RECOMENDADO



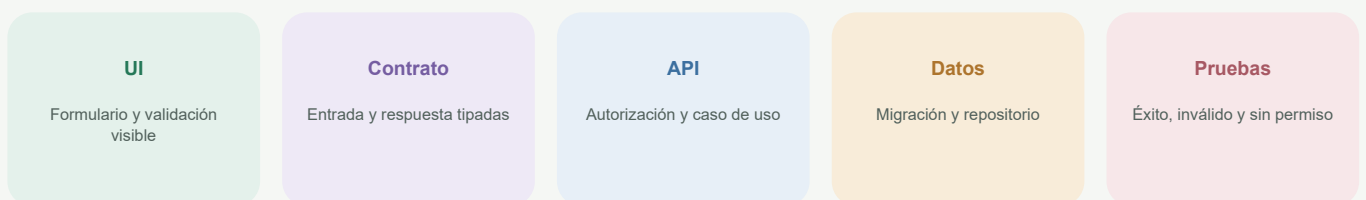
ANTES DE PROGRAMAR

- Requisito aprobado y trazable.
- Criterios de aceptación observables.
- Roles y permisos definidos.
- Datos de entrada y salida acordados.
- Riesgos y dependencias identificados.
- Diseño revisado por otra persona.

ESTADOS MÍNIMOS DE INTERFAZ

- | | |
|--------------------|-------------------------------|
| Vacío | qué ve alguien sin registros |
| Cargando | retroalimentación y bloqueo |
| Error | mensaje y forma de recuperar |
| Éxito | confirmación del resultado |
| Sin permiso | explicación sin filtrar datos |

EJEMPLO DE CORTE VERTICAL: CREAR UN PROYECTO



DEFINICIÓN DE TERMINADO

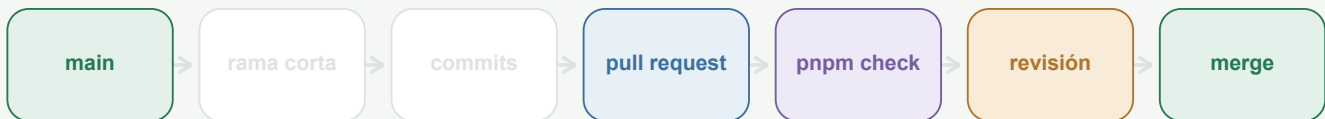
- Responde a un requisito o tarea identificable.
- Incluye pruebas proporcionales al riesgo.
- Supera pnpm check y una revisión cruzada.
- Valida entradas, permisos y errores.
- Actualiza contratos, documentación y ADR afectados.



Trabajo en equipo, Git y revisiones

Cada cambio debe ser pequeño, trazable y revisable. El repositorio ya incluye plantillas para issues, pull requests y CI; úsenlas como parte del proceso, no como decoración.

FLUJO GIT



NOMBRES Y MENSAJES

RAMAS

feature/descripcion
fix/descripcion
docs/descripcion
chore/descripcion

CONVENTIONAL COMMITS

feat: agrega ...
fix: valida ...
docs: registra ...
test: cubre ...
chore: actualiza ...

REVISIÓN CRUZADA POR RESPONSABILIDAD

Arquitectura

Límites, dependencias y ADR.

Análisis

Requisito, criterio y trazabilidad.

UI/UX

Interacción, accesibilidad y estados.

Desarrollo

Código, pruebas y operación.

NUNCA SUBIR

.env, tokens, claves, credenciales, datos personales, bases reales ni archivos de estudiantes. .env.example solo debe contener nombres y valores locales no sensibles.



Componentes futuros y decisiones pendientes

El enunciado preliminar propone diez componentes. El boceto actual los anticipa visualmente, pero ninguno está implementado como capacidad de negocio.

MAPA PRELIMINAR DE CAPACIDADES

ADMINISTRACIÓN

- Administración general
- Elementos del proyecto
- Tareas

DOCUMENTACIÓN

- Documentos del proyecto
- Generador PDF
- Bitácora

CONTROL TRANSVERSAL

- Alarmas
- Consistencia

INDICADORES

- Estadísticas
- Matrices de trazado

DECISIONES QUE REQUIEREN ADR

Persistencia Modelo, motor, ORM y migraciones.

Identidad SSO, sesiones y permisos por rol.

Documentos Editor enriquecido, imágenes y versiones.

PDF Motor HTML a PDF y plantillas.

Historial Inmutabilidad, recuperación y auditoría.

Integraciones Educandus, encuestas y donaciones.

Operación Despliegue, logs, respaldo y recuperación.

Privacidad Datos académicos, retención y acceso.

Orden recomendado: requisitos y permisos -> modelo de dominio -> persistencia -> primer corte vertical -> capacidades transversales -> integraciones externas.



Ruta de continuación y reglas no negociables

FUENTES DE VERDAD, DE MAYOR A MENOR PRIORIDAD

1. Enunciado vigente y aclaraciones oficiales**2. Plantillas y requisitos aprobados de la unidad****3. ADR y documentación técnica del equipo****4. Issues, pull requests y esta guía**

PRÓXIMOS SEIS HITOS

01 Asignar roles

Registrar responsables por unidad.

02 Cerrar requisitos

Completar plantilla y criterios.

03 Aprobar arquitectura

Resolver ADR prioritarios.

04 Diseñar el dominio

Entidades, permisos y contratos.

05 Construir cortes

Historias pequeñas de extremo a extremo.

06 Validar el 40%

Probar con usuarios y medir cobertura.

REGLAS NO NEGOCIABLES

- No programar requisitos ambiguos.
- No agregar dependencias sin justificar.
- No mezclar objetivos en un mismo PR.
- Registrar decisiones difíciles de revertir.
- No conectar la web directamente a datos.
- No guardar secretos en Git.
- No integrar con pnpm check en rojo.
- Actualizar documentación junto al código.

La guía evoluciona con el proyecto.

Cuando cambie el enunciado, la arquitectura o la navegación, actualicen el repositorio y este documento en el mismo pull request.